

## Sammlungen, Dateien, Rangliste

**Organisation:** Sie arbeiten im Zweierteam mit den Rollen A und B im bisherigen Repository. Wo eine Aufgabe die Rollen trennt, arbeitet jede Person in einem eigenen Branch und mergt nach umbau-sammlungen.

**Abgabe:** Tag abgabe-4 auf main, danach ein ZIP des Projektordners **mit** dem Ordner .git über Moodle/APA. Frist: [OFFEN: Datum]. Testat 4 verwendet die Klassennamen dieses Blatts.

Sie schreiben den Java-Code dieses Blatts ohne KI-Werkzeug und bedienen Git selbst, Befehl für Befehl. Im Testat und in der Vorführung erklären Sie jede Zeile und jeden Befehl.

Grundlage ist freiraum-start-4.zip aus Moodle: der Referenzstand nach Blatt 3, der Ordner daten/ mit drei CSV-Dateien und die Klasse Sortierung aus fremder Hand. Alle Räume, Gruppen und Zeiten sind frei erfunden; jede Datei beginnt mit einer Kommentarzeile, die das sagt.

**Erste Aufgabe (15 Minuten):** Spielen Sie den Startstand in einem Branch start-4 ein, wie es die README.md im ZIP beschreibt, oder übernehmen Sie zu Ihrem eigenen Stand nur daten/ und Sortierung.java. Mergen Sie nach main und legen Sie den Branch umbau-sammlungen an; er wird erst nach der siebten Aufgabe gemergt. Starten Sie PruefungKalender aus dem Projektordner, also dem Ordner mit pom.xml und daten/: Die letzte Zeile lautet 15 OK, 0 FEHLER. Öffnen Sie daten/buchungen-kaputt.csv im Editor. Die Zeilennummern des Editors zählen die zwei Kommentarzeilen mit; genau diese Nummern soll Ihr Ladebericht später nennen.

**Zweite Aufgabe (70 Minuten):** Buchungen kommen ab jetzt aus Dateien. raeume.csv hat die Spalten kennung;art;plaetze;einweisung mit den Arten lernraum, labor und projektraum. Die Buchungsdateien haben sieben Spalten raum;tag;beginn;ende;gruppe;personen;einweisung, zum Beispiel UX-LAB;1;12:00;14:00;Team Blau;1;ja. Zeilen, die mit # beginnen, und leere Zeilen sind keine Daten. Vorgegeben sind diese Typen und Köpfe:

```
public record Buchungszeile(String raum, Buchung buchung) { }
public record Fehlerzeile(int nummer, String grund) { }
public record Ladebericht(String datei, List<Buchungszeile> gueltige, List<Fehlerzeile> fehler) { }

// RaumDatei
public static Campus lade(Path datei) throws IOException
// BuchungsDatei
public static Buchungszeile zeileLesen(String zeile)
public static Ladebericht lade(Path datei, Campus campus) throws IOException
```

**(A)**RaumDatei.lade liest alle Zeilen mit Files.readAllLines und legt je Datenzeile einen Lernraum, ein Labor oder bei projektraum einen Raum an; einweisung ja ergibt die Regel NurMitEinweisung, sonst OhneBeschraenkung. Lernraum und Labor erhalten dafür einen Konstruktor mit Zugangsregel. Eine fehlerhafte Zeile in raeume.csv bricht das Laden mit einer IllegalArgumentException ab, die Dateiname und Zeilennummer nennt.

**(B)**BuchungsDatei.zeileLesen zerlegt eine Zeile an ;. Hat sie nicht genau sieben Spalten, wirft die Methode eine IllegalArgumentException mit dem Text 6 statt 7 Spalten (mit der tatsächlichen Anzahl). Das Zeitfenster entsteht mit Zeitfenster.aus, die Personenzahl mit Integer.parseInt; einweisung muss ja oder nein sein. Die Methode wirft, fängt aber nichts.

Gemeinsam schreiben Sie `BuchungsDatei.lade`. Die Zeilennummer ist der Index plus eins. Jede Datenzeile wird gelesen und in den Raum ihrer Kennung gebucht; eine Zeile, die scheitert, landet als Fehlerzeile im Bericht, und die nächste Zeile kommt dran. Die Gründe lauten unbekannter Raum LR-7 für eine Kennung, die der Campus nicht kennt, keine gültige Zahl ( mit der Meldung der `NumberFormatException` und ), bei jeder anderen `IllegalArgumentException` deren Meldung und bei `BuchungAbgelehntException` abgelehnt: mit dem Grund. `NumberFormatException` ist eine Unterklasse von `IllegalArgumentException`; ordnen Sie die `catch`-Blöcke entsprechend. Die `IOException` fängt `lade` nicht. Ergänzen Sie Fehlerzeile.toString() (Zeile 6: ...) und Ladebericht.asText(). Für `buchungen-kaputt.csv`, geladen in einen frisch aus `raeume.csv` gelesenen Campus, druckt `asText()`:

```
buchungen-kaputt.csv: 19 Buchungen übernommen, 4 Zeilen fehlerhaft
Zeile 6: Ende 12:30 liegt nicht nach Beginn 14:00
Zeile 11: keine gültige Zahl (For input string: "10")
Zeile 17: 6 statt 7 Spalten
Zeile 23: unbekannter Raum LR-7
```

Für `buchungen-woche.csv` lautet die Zeile `buchungen-woche.csv`: 40 Buchungen übernommen, 0 Zeilen fehlerhaft. Halten Sie beide Ergebnisse in `PruefungDatei` als `pruefe`-Fälle fest.

Das Buchungsterminal lädt beim Start `raeume.csv` und `buchungen-woche.csv` statt `Beispieldaten.campus()`. Zwei Proben: Streichen Sie im Terminal kurz das `try` um das Laden und lesen Sie `error: unreported exception IOException; must be caught or declared to be thrown`. Starten Sie `PruefungDatei` außerhalb des Projektordners; die Referenz bricht dann mit `Exception in thread "main" java.nio.file.NoSuchFileException: daten/raeume.csv` ab. Das Terminal meldet in diesem Fall, in welchem Ordner es gestartet werden muss, und beendet sich. Notieren Sie in `NOTIZEN.md` unter „Blatt 4“, wer die `IOException` fängt und warum nicht `lade` selbst.

**Dritte Aufgabe (60 Minuten):** Die eigenen Behälter aus Blatt 1 bis 3 weichen den Sammlungen aus `java.util`. Die öffentlichen Methoden behalten Namen und Wirkung; die Prüfprogramme laufen danach ohne FEHLER.

- (A) Campus hält `Map<String, Raum>` als `LinkedHashMap`, damit die Räume in der Reihenfolge des Hinzufügens bleiben; finde fragt die Map. Neu sind `List<Raum> alle()` und `List<Buchungszeile> buchungszeilen()` über alle Räume. Raum hält `List<Buchung>` statt des Arrays; die Grenze von 20 Buchungen und ihr Ablehnungsgrund entfallen. Die Warteliste wird `Deque<Buchung>` (`ArrayDeque`); Warteliste und Eintrag löschen Sie. `stornieren` und das Entfernen aus der Warteliste arbeiten mit einem Iterator und `it.remove()`. Probieren Sie vorher einmal, in einer `for-each`-Schleife zu entfernen, und tragen Sie die erste Zeile der Meldung (`java.util.ConcurrentModificationException`) mit Stacktrace-Anfang wörtlich in `NOTIZEN.md` ein. Schreiben Sie `PruefungWarteliste` auf die Wartelisten-Methoden von Raum um.
- (B) Belegungskalender hält `TreeMap<Integer, List<Buchung>>` und bietet weiter `einfuegen`, `enthaeltStart`, `zaehle`, `frueheste` und `ausgebenSortiert`. Kopieren Sie Ihren Baum aus Blatt 3 vorher in eine Klasse `BaumKalender`; die vierte Aufgabe misst beide gegeneinander. Bauen Sie den Kalender zuerst mit `TreeMap<Integer, Buchung>` und fügen Sie `Beispieldaten.woche()` ein: Die Referenz zählt danach 34 statt 40 Buchungen. Schreiben Sie in `NOTIZEN.md`, welche sechs fehlen und warum, und stellen Sie dann auf `List<Buchung>` als Wert um.

Schreiben Sie `PruefungSammlungen` mit mindestens diesen Fällen: unbekannte Kennung liefert null; eine doppelte Kennung wird abgewiesen; ein Raum nimmt 25 Buchungen an; zwei Buchungen in verschiedenen Räumen mit gleicher Startzeit ergeben im Kalender `zaehle()` 2. Bekommen Sie beim Kompilieren `incompatible types: Object cannot be converted to Buchung`, fehlt irgendwo `<Buchung>` hinter `List`.

**Vierte Aufgabe (30 Minuten):** Schreiben Sie `KalenderMessung`. Die `main` fügt `BeispielDaten.semesterplan()` in einen `BaumKalender` und in einen `Belegungskalender` ein und fragt danach beide 1000-mal `enthaeltStart` für Freitag 21:15 ( $5 * 1440 + 1275$ ). Messen Sie jede der vier Phasen mit `System.nanoTime()` und geben Sie Millisekunden aus. Wiederholen Sie die Messung in fünf Runden in derselben `main`; die ersten Runden sind oft langsamer. Im Referenzlauf braucht der eigene Baum einige hundert Millisekunden, die `TreeMap` wenige. Fügen Sie den Plan danach ein zweites Mal in einen `BaumKalender` ein, diesmal gemischt mit `Collections.shuffle(liste, new Random(BeispielDaten.SAMEN))`, und geben Sie die Höhe aus; die Referenz ergibt 76.

Bricht die Messung mit `java.lang.StackOverflowError` ab, ist das Ihr Messwert für den eigenen Baum: Mit 1 MB Stack scheitert die Referenzmessung in `BaumKalender.enthaeltStart` [PRÜFEN: Laufprobe unter Windows]. Tragen Sie die Zeiten oder das Fehlerbild in `NOTIZEN.md` ein, dazu Ihre Entscheidung: Welcher Behälter trägt den Kalender, nach welchem Kriterium, und welchen Preis zahlen Sie?

**Fünfte Aufgabe (40 Minuten):** `Sortierung.einfuegeSortieren` stammt aus fremder Hand und ist nicht geprüft. `Raum.belegungAusgeben()` soll damit die Buchungen eines Raums chronologisch ausgeben. Prüfen Sie den Code, bevor Sie ihm vertrauen:

1. Schreiben Sie `PruefungSortierung`, ohne den Code zu lesen. Übertragen Sie die Batterie aus T09 auf das Sortieren: leere Liste, ein Element, zwei Elemente falsch herum, schon sortiert, umgekehrt sortiert, kleinstes Element vorn, kleinstes Element hinten, zwei Buchungen mit gleicher Startzeit (ihre Reihenfolge bleibt), und alle Buchungen aus `woche()` gegen dieselbe Liste nach `liste.sort(Sortierung.CHRONOLOGISCH)`.
2. Starten Sie das Programm und notieren Sie, welche Fälle FEHLER melden.
3. Lesen Sie jetzt den Code mit der Invariante aus seinem Kommentar. Suchen Sie die Zeile, nach der die Invariante nicht mehr gilt, und den kleinsten Testfall, der den Fehler zeigt.
4. Korrigieren Sie die Zeile und starten Sie die Prüfung erneut, bis alle Fälle OK melden.

In `NOTIZEN.md` stehen danach der kleinste Testfall mit erwartetem und tatsächlichem Ergebnis, die fehlerhafte Zeile und der Prüfschritt, mit dem Sie den Fehler auch ohne Lauf gefunden hätten.

**Sechste Aufgabe (50 Minuten):** Die Rangliste ordnet die Räume nach ihrer Auslastung. Vorgegeben ist `public record Auslastung(String raum, int minuten) { }`. `Rangliste.berechne(List<Buchungszeile> zeilen)` liefert die sortierte `List<Auslastung>`, `Rangliste.asText(...)` druckt je Platz eine Zeile mit `String.format("%d. %-10s %5d Minuten%n", platz, raum, minuten)`.

(A) Summieren Sie die Minuten je Raum in einer `Map<String, Integer>`, die die Reihenfolge des ersten Auftretens behält, und erzeugen Sie daraus die Liste.

(B) Schreiben Sie den `Comparator<Auslastung>` REGEL: absteigend nach Minuten, bei Gleichstand aufsteigend nach Kennung.

Sortieren Sie die Buchungen aus `buchungen-woche.csv` zuerst nur nach Minuten mit `(a, b) -> b.minuten() - a.minuten()`. Die Referenz druckt:

|              |              |
|--------------|--------------|
| 1. PR-1      | 1590 Minuten |
| 2. UX-LAB    | 1380 Minuten |
| 3. LR-1      | 1140 Minuten |
| 4. LR-2      | 960 Minuten  |
| 5. AUDIO-LAB | 960 Minuten  |

Mit REGEL druckt sie:

|              |              |
|--------------|--------------|
| 1. PR-1      | 1590 Minuten |
| 2. UX-LAB    | 1380 Minuten |
| 3. LR-1      | 1140 Minuten |
| 4. AUDIO-LAB | 960 Minuten  |
| 5. LR-2      | 960 Minuten  |

Schreiben Sie in NOTIZEN.md, warum LR-2 ohne Regel vorn steht, obwohl niemand das festgelegt hat, und warum T09 Integer.compare statt der Subtraktion empfiehlt. Das Terminal erhält den Menüpunkt 5 Rangliste.

**Siebte Aufgabe (35 Minuten):** Schreiben Sie PruefungRangliste mit diesen Fällen: die erwartete Rangliste für buchungen-woche.csv; zwei Räume mit gleichen Minuten in umgekehrter Reihenfolge der Kennung; eine leere Liste; eine einzelne Buchung; zwei Buchungen im selben Raum, die summiert werden; der unbekannte Raum LR-7 aus buchungen-kaputt.csv erscheint nicht in der Rangliste. Laufen alle Prüfprogramme ohne FEHLER, mergen Sie umbau-sammlungen in main, setzen Sie den Tag abgabe-4 und pushen Sie Branch und Tag.